Task 02: Uncertainty-Aware CNN Extension

Akram Aki

May 4, 2026

1. Uncertainty-Aware CNN Extension

This report section extends the previous GALAH spectra regression workflow. The data download, loading, preprocessing, normalization, dataset class, dataloaders, and device setup were kept the same as in the previous task. In particular, the same normalized one-dimensional stellar spectra were used as inputs, and the model still predicts the three stellar parameters T_{eff}, $\log g$, and [Fe/H]. The main change is that the standard regression CNN was replaced by an uncertainty-aware one-dimensional CNN.
Instead of predicting only one value per stellar parameter, the network predicts both a mean value and an uncertainty. The output is split into

$$\mu \in \mathbb{R}^{B\times 3} \qquad \text{and} \qquad \log\sigma^2 \in \mathbb{R}^{B\times 3},$$

where B is the batch size and the three columns correspond to T_{eff}, $\log g$, and [Fe/H]. Predicting the logarithm of the variance is numerically more stable than predicting the variance directly. It also guarantees a positive variance after exponentiation,

$$\sigma^2 = \exp(\log\sigma^2).$$

2. Gaussian Negative Log-Likelihood Loss

The model was trained with a Gaussian negative log-likelihood loss. For each target value y, predicted mean μ, and predicted log variance $\log\sigma^2$, the loss is

$$\mathcal{L} = \frac{1}{2}\left(\log\sigma^2 + \frac{(y-\mu)^2}{\exp(\log\sigma^2)}\right),$$

followed by averaging over the batch and the three output parameters:

$$\mathcal{L}_{\text{final}} = \text{mean}(\mathcal{L}).$$

This loss penalizes inaccurate predictions through the squared residual term. At the same time, it prevents the model from simply predicting very large uncertainties, because the $\log\sigma^2$ term increases when the predicted variance becomes too large. Therefore, the model must learn both accurate mean predictions and meaningful uncertainty estimates.

3. Model Configurations and Hyperparameter Sweep

Several uncertainty-aware CNN configurations were trained as a small controlled hyperparameter optimization. The goal was not to perform a large automated search, but to compare a limited number of reasonable CNN configurations and select a model with good prediction performance and well-calibrated uncertainties. Calibration was evaluated especially through the pull distributions.
The exact model configurations are provided in the appendix in code form. The experiments should be reproducible from the accompanying GitHub repository, using the provided model configurations. As in the previous report, the GitHub repository is publicly available at:

`https://github.com/AkramAki/Advanced_DeepLearning_Seminar`

Since this report corresponds to the state of the work on 4 May 2026, the repository version or commit from 4 May 2026 should be used for reproducibility if later changes are made.

Run name	Best validation loss	Best epoch
`k3_c16-32-64_drop0.0_bn`	-1.388008	37
`k9_c16-32-64_1conv_drop0.2_no_bn`	-1.356892	29
`k3_c16-32-64_drop0.1_bn`	-1.214014	31
`k5_c16-32-64_drop0.1_bn`	-1.139896	16
`k3_c32-64-128_drop0.1_bn`	-1.112192	17
`k3_c16-32-64-128_drop0.1_pool32_bn`	-1.058081	23

Table 1: Summary of the uncertainty-aware CNN model configurations. The full configurations are listed in the appendix.

4. Results

Figure 1 shows the Gaussian negative log-likelihood loss during training. The plot contains the training and validation losses as a function of epoch, together with the test loss and the epoch of the best checkpoint. The validation curve was used to select the best model checkpoint. Overall, the loss curves show whether the uncertainty-aware model converged stably and whether the validation loss followed the training loss without strong divergence.
The predicted-versus-true comparison is shown in Figure 2. The model predicts the mean stellar parameters, while the predicted uncertainties are shown as error bars. Most points are expected to lie close to the one-to-one relation if the mean predictions are accurate. The error bars provide additional information about the model's confidence in each prediction. Compared with a standard regression CNN, this is a more informative output because the model does not only return a point estimate, but also an uncertainty estimate for each stellar parameter.

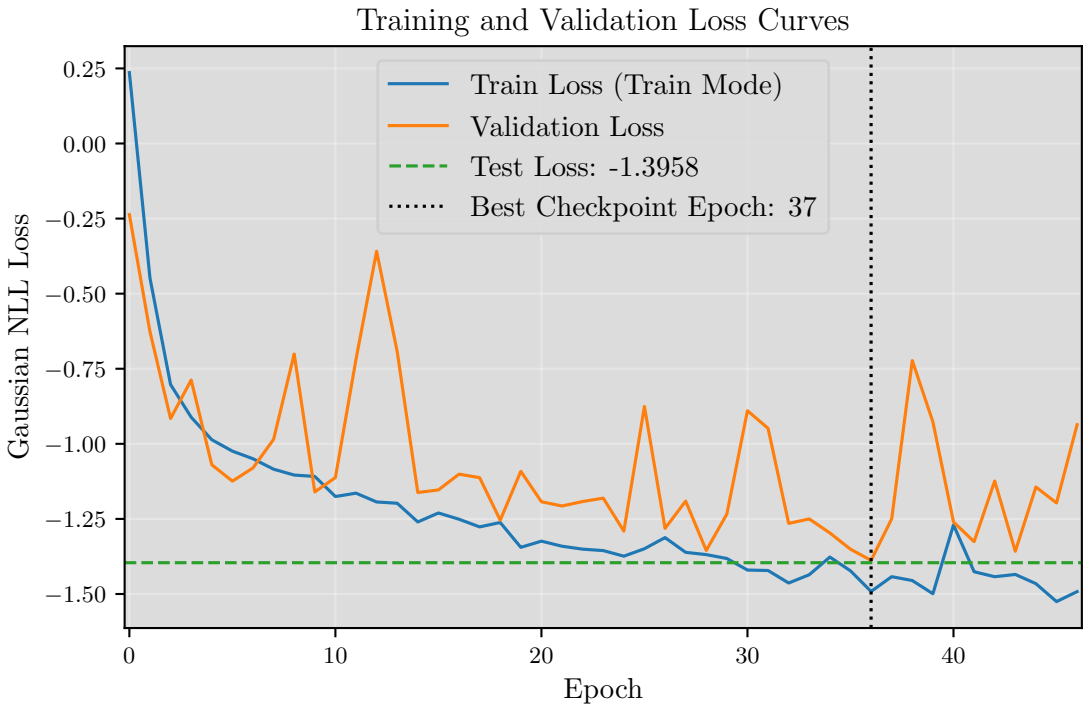

Figure 1: Training and validation Gaussian NLL loss curves. The test loss and best checkpoint epoch are shown in the figure.

The uncertainty calibration was evaluated using pull distributions, shown in Figure 3. The pull is defined as

$$\text{pull} = \frac{\text{truth} - \text{prediction}}{\sigma}$$

where σ is the predicted standard deviation. For a well-calibrated uncertainty model, the pull distribution should have a mean close to zero and a standard deviation close to one. A shifted pull distribution indicates bias in the predictions, while a width different from one indicates uncertainty miscalibration.

The final pull statistics are summarized in Table 2. The T_{eff} pull distribution has a mean of -0.004 and a standard deviation of 1.011, which is very close to the ideal values of zero and one. This indicates that the T_{eff} uncertainty estimates are very well calibrated. For $\log g$, the pull mean is -0.093 and the pull standard deviation is 1.095. The small negative mean indicates a slight bias, while the width larger than one means that the model is slightly overconfident for this parameter. In other words, the predicted uncertainties for $\log g$ are slightly too small compared with the actual residuals.

For [Fe/H], the pull mean is -0.017 and the pull standard deviation is 0.903. The mean is close to zero, so there is little bias. However, the width is smaller than one, indicating that the model is slightly underconfident for metallicity, meaning that the predicted uncertainties are somewhat larger than necessary.

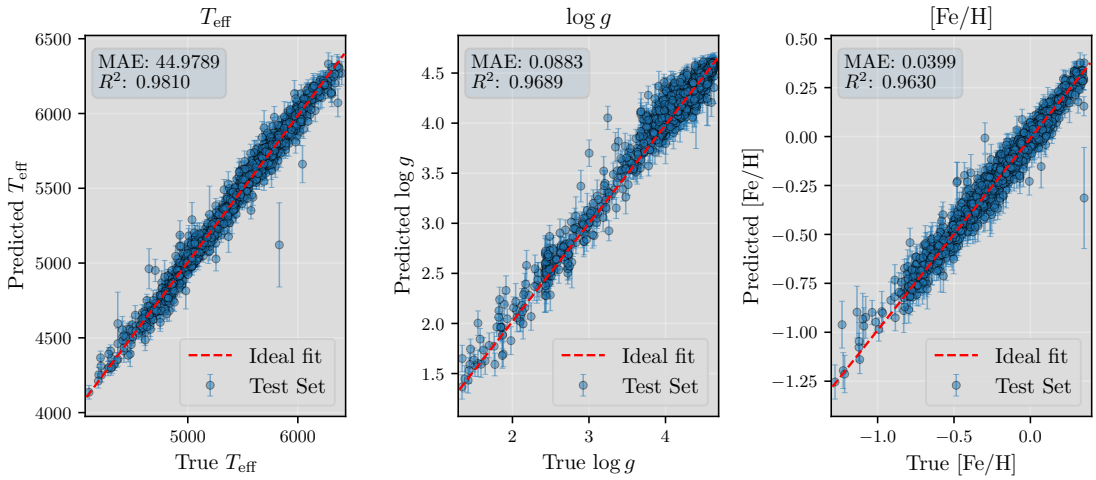

Figure 2: Predicted versus true stellar parameters on the test set. The error bars show the predicted uncertainties.

Parameter	Pull mean	Pull standard deviation
$T_{\rm eff}$	-0.004	1.011
$\log g$	-0.093	1.095
[Fe/H]	-0.017	0.903

Table 2: Final pull statistics for the uncertainty-aware CNN on the test set.

Overall, the uncertainty-aware CNN produces reasonably well-calibrated uncertainty estimates. The best calibrated parameter is $T_{\rm eff}$, whose pull distribution is almost ideal. The $\log g$ uncertainties are slightly overconfident, while the [Fe/H] uncertainties are slightly underconfident. Nevertheless, all three pull distributions are close to the desired behavior, showing that the Gaussian NLL training objective successfully encouraged the model to learn useful predictive uncertainties in addition to accurate mean predictions.

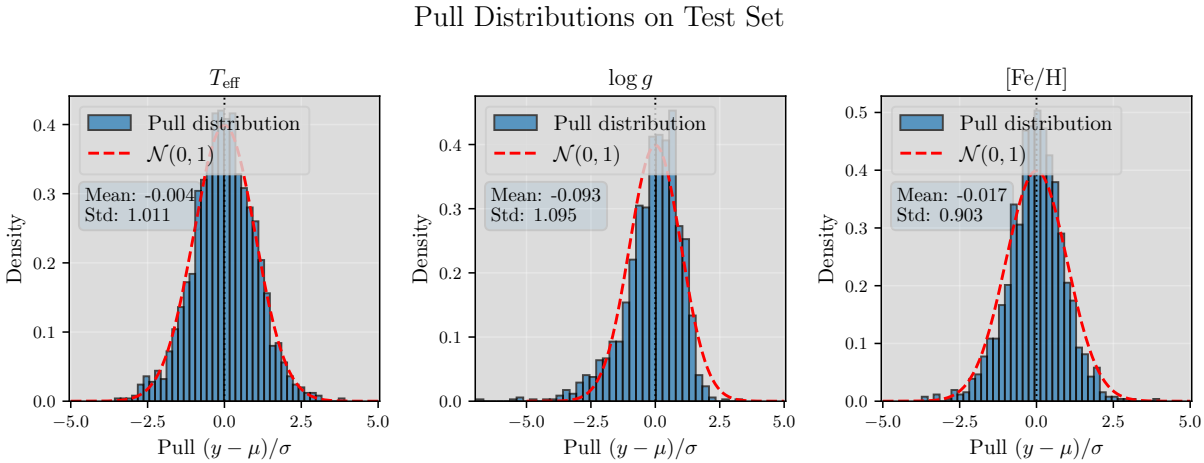

Figure 3: Pull distributions for $T_{\rm eff}$, $\log g$, and [Fe/H]. A well-calibrated model should have pull mean close to zero and pull standard deviation close to one.

5. Conclusion

This extension replaced the standard CNN regression model with an uncertainty-aware CNN that predicts both mean stellar parameters and log variances. The same data pipeline as in the previous report was used, so the main methodological difference was the model output and the Gaussian negative log-likelihood loss. The small hyperparameter sweep allowed several CNN configurations to be compared in a controlled way. The final model achieved reasonable uncertainty calibration, with very good calibration for $T_{\rm eff}$, slight overconfidence for $\log g$, and slight underconfidence for [Fe/H].

A. Model Configurations

The uncertainty-aware CNN models used in the sweep were defined by the following configurations:

```
model_configs = [
{
"run_name": "k3_c16-32-64_drop0.1_bn",
"channels": (16, 32, 64), "kernel_size": 3,
"convs_per_block": 2, "dropout": 0.1,
"adaptive_pool_length": 64, "hidden_dim": 128,
"use_batchnorm": True,
},
{
"run_name": "k3_c16-32-64_drop0.0_bn",
"channels": (16, 32, 64), "kernel_size": 3,
"convs_per_block": 2, "dropout": 0.0,
"adaptive_pool_length": 64, "hidden_dim": 128,
"use_batchnorm": True,
```

```
},
{
"run_name": "k5_c16-32-64_drop0.1_bn",
"channels": (16, 32, 64), "kernel_size": 5,
"convs_per_block": 2, "dropout": 0.1,
"adaptive_pool_length": 64, "hidden_dim": 128,
"use_batchnorm": True,
},
{
"run_name": "k3_c32-64-128_drop0.1_bn",
"channels": (32, 64, 128), "kernel_size": 3,
"convs_per_block": 2, "dropout": 0.1,
"adaptive_pool_length": 64, "hidden_dim": 128,
"use_batchnorm": True,
},
{
"run_name": "k3_c16-32-64-128_drop0.1_pool32_bn",
"channels": (16, 32, 64, 128), "kernel_size": 3,
"convs_per_block": 2, "dropout": 0.1,
"adaptive_pool_length": 32, "hidden_dim": 128,
"use_batchnorm": True,
},
{
"run_name": "k9_c16-32-64_1conv_drop0.2_no_bn",
"channels": (16, 32, 64), "kernel_size": 9,
"convs_per_block": 1, "dropout": 0.2,
"adaptive_pool_length": 64, "hidden_dim": 128,
"use_batchnorm": False,
},
]
```


B. Uncertainty-Aware CNN Architecture

The model is a configurable one-dimensional CNN with convolutional blocks, adaptive average pooling, a shared fully connected head, and two output heads. One head predicts the mean stellar parameters, while the second predicts the corresponding log variances. Each convolutional block applies one or more 1D convolutions, optional batch normalization, ReLU activations, and max pooling. The number of channels, kernel size, number of convolutions per block, dropout rate, and adaptive pooling length are set by the configuration.
The general structure is

$$x \to \text{CNN Blocks} \to \text{AdaptiveAvgPool1d} \to \text{Flatten} \to \text{Shared Head} \to (\mu, \log \sigma^2).$$

The shared head maps the extracted spectral features to a hidden representation,

$$h = \text{Dropout}\left(\text{ReLU}\left(W_{\text{shared}} f + b_{\text{shared}}\right)\right),$$

where f is the flattened feature vector after convolution and pooling.

The two output heads then predict

$$\mu = W_\mu h + b_\mu,$$

$$\log \sigma^2 = W_{\log \sigma^2} h + b_{\log \sigma^2}.$$

For numerical stability, the predicted log variance is clamped:

$$\log \sigma^2 = \text{clamp}\left(\log \sigma^2, -10, 10\right).$$

The log-variance head was initialized with zero weights and bias, giving an initial log variance close to zero, i.e. variance close to one in normalized label space. The forward pass returns

$$\text{model}(x) = \left(\mu, \log \sigma^2\right).$$